

ACID Transactions

18 August 2026 01:02 PM

ACID Transactions

ACID gives databases guarantees that make transactions **reliable and safe**.

- **A — Atomicity:** All changes happen, or none happen.
- **C — Consistency:** Data remains valid according to defined rules.
- **I — Isolation:** Concurrent transactions don't interfere in unsafe ways.
- **D — Durability:** Once committed, data survives a crash.

ACID = Reliable transactions

1. What a Transaction Looks Like

A **transaction** is a group of database operations treated as **one unit**.

Example: Transfer ₹100 from Account A to Account B.

BEGIN

↓

Debit A ₹100

↓

Credit B ₹100

↓

COMMIT

If something fails:

BEGIN

↓

Debit A ₹100

↓

✗ Error

↓

ROLLBACK

ROLLBACK removes the changes made inside the transaction.

2. Atomicity

Atomicity = All or Nothing

Suppose:

Account A = ₹1000

Account B = ₹500

Transfer ₹100:

A → -₹100

B → +₹100

Both operations must happen.

Without Atomicity ✗

A → ₹900

B → ₹500

Money disappeared.

With Atomicity ☑

Either:

A → ₹900

B → ₹600

or if something fails:

A → ₹1000

B → ₹500

Nothing is partially saved.

Remember

Atomicity prevents half-completed transactions.

3. Consistency

Consistency = Data must follow the rules.

The database can enforce rules such as:

- Primary key must be unique
- Foreign key must reference a valid row
- NOT NULL values cannot be empty
- CHECK constraints must be satisfied
- Data types must be valid

Example:

```
CREATE TABLE accounts (  
id INT PRIMARY KEY,  
balance DECIMAL(10,2)  
CHECK (balance >= 0)  
);
```

This prevents:

balance = -500

because the database rule says the balance cannot be negative.

Important

The database cannot automatically understand **every business rule**.

For example:

Payment System

↓

Inventory System

↓

Shipping System

The database doesn't automatically know that all three systems must agree about an order.

Those rules may require:

- Application logic
- Transactions
- Background jobs
- Retries
- Error handling
- Cross-system workflows

Remember

Consistency = After a transaction, the data must still follow the rules.

4. Isolation

Isolation controls how multiple transactions behave when they run **at the same time**.

In real systems, many transactions happen simultaneously:

User A → Buy product

User B → Buy same product

Worker → Update inventory

Isolation prevents these transactions from interfering with each other in unsafe ways.

Common Concurrency Problems

1. Dirty Read

A transaction reads data that another transaction has **changed but not committed yet**.

Transaction A → Changes balance to ₹500

Transaction B → Reads ₹500

Transaction A → ROLLBACK

Transaction B read data that was never actually committed.

Dirty Read = Reading uncommitted data

2. Non-Repeatable Read

A transaction reads the **same row twice** but gets different values.

Transaction A → Reads balance = ₹1000

Transaction B → Updates balance = ₹800

→ COMMIT

Transaction A → Reads balance again = ₹800

Same transaction, same row, different value.

Non-repeatable read = Same row gives different values

3. Phantom Read

A transaction runs the same query twice but gets a **different number of rows**.

Example:

```
SELECT * FROM products
```

```
WHERE price < 1000;
```

First time:

5 products

Another transaction inserts a matching product.

Second time:

6 products

The new row is called a **phantom row**.

Phantom Read = New matching rows appear/disappear

4. Lost Update

Two transactions update the same data and one update **overwrites the other**.

A reads balance = 1000

B reads balance = 1000

A updates → 900

B updates → 800

A's update is lost.

Lost Update = One transaction overwrites another transaction's change

5. Write Skew

Two transactions read related data and independently make changes that **together break a rule**.

Write Skew = Individual changes look valid, but together they violate a business rule.

Isolation Levels

Most relational databases let you choose an isolation level.

Higher isolation protects you from more concurrency bugs. The tradeoff is that transactions may wait longer, conflict more often, or need retries.

Level	Simple Meaning
Read Uncommitted	Can read uncommitted changes
Read Committed	Prevents dirty reads
Repeatable Read	Protects repeated reads better
Serializable	Transactions behave more like running one at a time

Important

Higher isolation gives **more protection**, but can also cause:

- More waiting
- More conflicts
- More retries
- Lower concurrency

Higher isolation = Safer, but potentially slower.

Example: Overselling Inventory

Suppose:

Stock = 1

Two users purchase at the same time:

User A → Reads stock = 1

User B → Reads stock = 1

Both think the product is available.

User A → Buys it ☒

User B → Buys it ☒

Now you sold **2 items** when only **1** existed.

Possible solutions

You can use:

1. Conditional update

UPDATE products

SET stock = stock - 1

WHERE id = 10

AND stock > 0;

2. Row locking

SELECT ...

FOR UPDATE;

3. Unique reservation

Create a reservation that prevents the same item from being reserved twice.

4. Serializable isolation

Use stronger isolation and retry transactions when conflicts occur.

Isolation is a design decision, not simply a database setting you choose once.

How Databases Implement Isolation

Databases commonly use:

Locks

One transaction locks data while changing it.

Transaction A

↓

Lock Row

↓

Transaction B → Wait

MVCC

Multi-Version Concurrency Control keeps different versions of rows so readers can work with a stable snapshot while writers create newer versions.

Range Locks

Protect a **range of rows**, not just one row.

Example:

All products where price < ₹1000

Conflict Detection

The database detects that two transactions cannot safely run together and stops one transaction.

5. Durability

Durability = Once a transaction is committed, the data should survive a database failure, within the failure guarantees the database provides.

Example:

Payment saved

↓

COMMIT ☒

↓

Database crashes

↓

Database recovers

↓

Payment still exists ☒

Durability means committed data isn't simply lost because of a normal crash.

Write-Ahead Loggin

Many databases use a write-ahead log, or WAL.

The rule is simple:

Write the recovery record before relying on the changed data page.

The database may not immediately write every changed table page to its main data files. That would be slow. Instead, it first writes enough information to the WAL.

If the database crashes after commit but before the changed pages reach the main files, recovery can replay the WAL and restore the committed changes.

Durability Settings

Durability can involve a **speed vs safety trade-off**.

For example:

More durability

↓

More disk flushing

↓

More safety

↓

Potentially slower

Some systems may choose less strict durability for data that can easily be rebuilt, such as:

- Caches
- Analytics buffers
- Temporary data

But stronger durability is generally important for things like:

- Payments
- Orders
- User identity records
- Compliance data

Replication vs Durability

Replication can help when a database server fails:

Primary

↓

Replica

↓

Replica

But **asynchronous replication** may still lose recently acknowledged writes if the primary fails before replicas receive them.

Backups are also important for:

- Accidental deletion
- Corruption
- Human mistakes
- Disaster recovery

But backups are **not the normal transaction commit path**.